Mergesort bottom-up × Quicksort

Plano da apresentação — divisão do trabalho entre 5 pessoas · 15 minutos

O trabalho: implementar Mergesort e Quicksort em C++, medir o tempo de cada um com vetores de **30, 300, 3.000, 30.000 e 300.000** inteiros aleatórios, **20 execuções por tamanho** (5 × 20 × 2 = **200 ordenações**) e apresentar a comparação em 15 minutos. Já está tudo pronto na pasta `trabalho_ordenacao/`: o código C++, o painel `PAINEL_ADM.bat` (compila e roda tudo; precisa do Dev-C++ ou MinGW) e os slides (`apresentacao/slides.pptx`, com o roteiro de cada um nas anotações). O que falta é estudar a sua parte, colocar os nomes e ensaiar.

Como preencher: escrevam o nome de cada integrante na coluna “Responsável”. Cada pessoa fala ~2,5 a 3 min e prepara os próprios slides. Os tempos já somam 15:00.

Cronograma da apresentação

Tempo	Bloco	Responsável	Slides
0:00 – 3:00	**1. Abertura, como comparar e como medimos**	Pessoa 1: _____	1 – 3
3:00 – 5:30	**2. Mergesort bottom-up**	Pessoa 2: _____	4
5:30 – 8:30	**3. Quicksort clássico e complexidade**	Pessoa 3: _____	5 – 6
8:30 – 11:30	**4. Resultados, estatística descritiva e boxplot**	Pessoa 4: _____	7 – 9
11:30 – 14:00	**5. Crescimento, outras entradas e conclusão**	Pessoa 5: _____	10 – 13
14:00 – 15:00	Perguntas (cada um responde sobre a sua parte)	Todos	13

Linha condutora (o que a plateia deve levar para casa)

- Até 3.000 dados os dois algoritmos empatam: as diferenças ficam dentro da variação das medições.
- Com 30.000 e 300.000 dados o Quicksort clássico foi cerca de 9% e 15% mais rápido, porque troca elementos no próprio vetor, enquanto o Mergesort copia tudo para um vetor auxiliar a cada passada.
- Quando os dados crescem 10×, o tempo cresceu de 11× a 15×: o comportamento n log n previsto pela teoria.
- O Mergesort ganha em garantia (O(n log n) sempre) e estabilidade; o Quicksort depende do pivô, e o pivô do meio evitou o pior caso em vetores já ordenados.

O que cada pessoa faz

Pessoa 1 — Abertura, como comparar e como medimos

0:00 – 3:00 · slides 1 a 3

O que apresentar

- **Slide 1:** apresentar o grupo e o tema: Mergesort bottom-up × Quicksort clássico em C++.
- **Slide 2:** 5 tamanhos (30 a 300.000), 20 amostras cada, mesmos dados para os dois: 200 ordenações.
- **Slide 3:** passo a passo da medição e ferramentas (C++, Dev-C++, PAINEL_ADM.bat, Python).

Antes da apresentação

- Testar o `PAINEL_ADM.bat` no computador da apresentação (opções [2] e [11]) e levar um print de reserva.
- Controlar o tempo no ensaio (cada bloco tem horário de início na tabela da página 1).

Pessoa 2 — Mergesort bottom-up

3:00 – 5:30 · slide 4

O que apresentar

- **Slide 4:** blocos de 1 já estão ordenados; intercala blocos vizinhos de 2, 4, 8... Com 9 números são 4 passadas; com 300.000, 19. Custo n log n sempre, estável, usa vetor auxiliar.

Antes da apresentação

- Ler o material de estudo `apresentacao/estudo_mergesort_bottom_up.pdf` (painel, opção [10]).
- Preparar resposta para: “qual a diferença entre bottom-up e top-down?”.

Pessoa 3 — Quicksort clássico e complexidade

5:30 – 8:30 · slides 5 e 6

O que apresentar

- **Slide 5:** pivô = elemento do meio (6); dois índices trocam quem está do lado errado; resulta em [5 3 2 1 4] e [6 8 9 7]; repete em cada lado.
- **Slide 6:** n log n contra n^2: com 300.000 itens, milhões contra bilhões de passos.

Antes da apresentação

- Estudar a função `quickRec` em `src/benchmark.cpp`.
- Preparar resposta para: “o que acontece se o pivô for sempre o menor elemento?”.

Pessoa 4 — Resultados, estatística e boxplot

8:30 – 11:30 · slides 7 a 9

O que apresentar

- **Slide 7:** tabela de médias: empate até 3.000; Quick mais rápido em 30.000 e 300.000.
- **Slide 8:** estatística descritiva de 300.000: média, mediana, desvio, variância, CV e máximo/mínimo.
- **Slide 9:** boxplot: caixas sobrepostas = empate; caixas separadas = diferença real.

Antes da apresentação

- Saber explicar mediana, desvio padrão, coeficiente de variação e quartis (está no artigo, seção 2.4).
- Se o grupo rodar no próprio PC antes, comentar se os números de lá seguem o mesmo padrão.

Pessoa 5 — Crescimento, outras entradas e conclusão

11:30 – 14:00 · slides 10 a 13

O que apresentar

- **Slide 10:** dados ×10 → tempo ×11 a ×15, perto do previsto por n log n; n^2 seria ×100.
- **Slide 11:** com entradas ordenadas, invertidas ou repetidas o Quick foi 45% a 55% mais rápido.
- **Slides 12 e 13:** vantagens e desvantagens, qual escolher, agradecimento e perguntas.

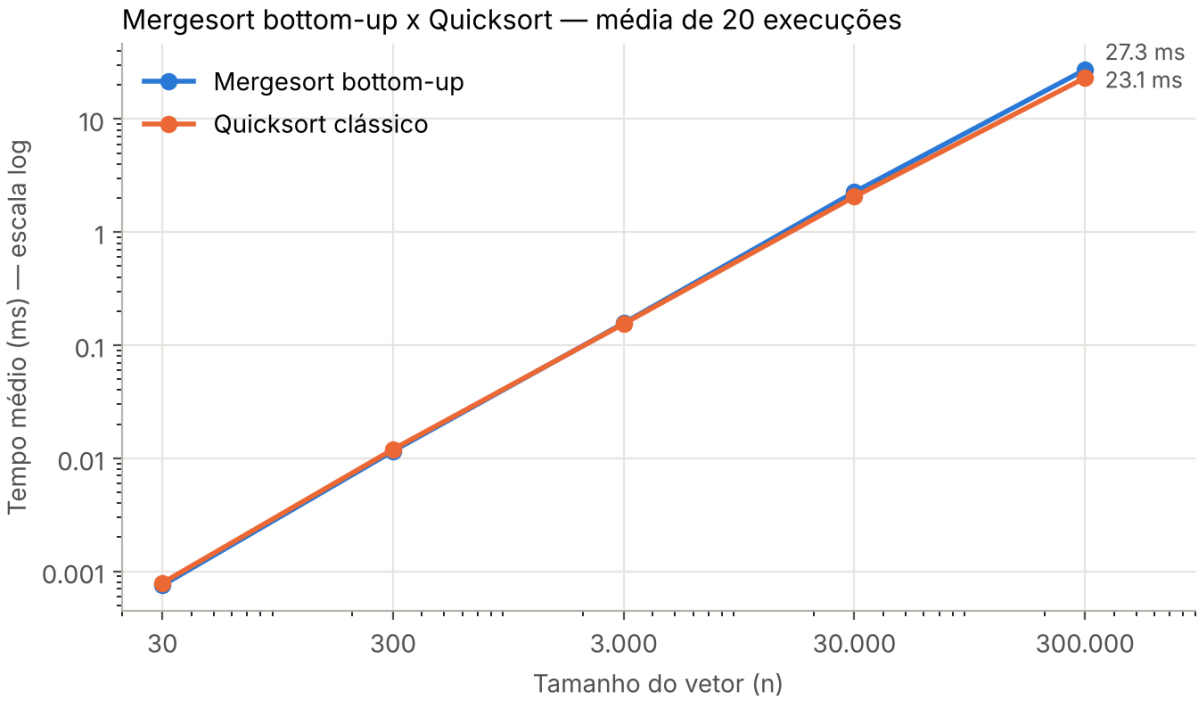
Antes da apresentação

- Conduzir a rodada de perguntas (14:00 a 15:00).
- Ler a Conclusão do artigo.

Resultados já medidos (slides 7 a 10)

Média e desvio padrão de 20 execuções, em milissegundos. Medido em um Intel Xeon 2,8 GHz; rodem de novo na máquina de vocês (PAINEL_ADM.bat, opção [2]). Os números mudam, o padrão se mantém.

n	Mergesort bottom-up (ms)	Quicksort clássico (ms)	Quicksort vs. Mergesort
30	0,0008 ± 0,0000	0,0008 ± 0,0000	5% mais lento
300	0,0116 ± 0,0003	0,0120 ± 0,0008	4% mais lento
3.000	0,1567 ± 0,0023	0,1545 ± 0,0082	empate técnico
30.000	2,26 ± 0,2033	2,06 ± 0,1641	9% mais rápido
300.000	27,28 ± 2,99	23,14 ± 2,89	15% mais rápido



Leitura rápida: de 30.000 para 300.000 (10× mais dados) o tempo cresceu de 11× a 12×, perto do previsto por n log n (10 × log 300.000 / log 30.000 ≈ 12,2). Um algoritmo O(n²) cresceria 100×.

Checklist do grupo

	Tarefa	Quem
☐	Preencher os nomes na tabela da página 1 e combinar o prazo dos slides	Todos
☐	Conferir os nomes no slide 1 e no artigo	Pessoa 1
☐	Teste final rodado, CSVs e gráfico atualizados	Pessoas 4 e 5
☐	Cada um leu as anotações do apresentador dos seus slides	Todos
☐	Ensaio completo cronometrado (meta: 13:30 sem as perguntas)	Todos
☐	Demo testada no computador da apresentação + plano B (print/vídeo)	Pessoa 4